



CS 332: Algorithms

Graph Algorithms

<https://www.cs.virginia.edu/~luebke/cs332.fall00/lecture18.ppt>

Review: Graphs

- A graph $G = (V, E)$
 - V = set of vertices, E = set of edges
 - *Dense* graph: $|E| \approx |V|^2$; *Sparse* graph: $|E| \approx |V|$
 - *Undirected graph*:
 - Edge (u,v) = edge (v,u)
 - No self-loops
 - *Directed* graph:
 - Edge (u,v) goes from vertex u to vertex v , notated $u \rightarrow v$
 - A *weighted graph* associates weights with either the edges or the vertices

Review: Representing Graphs

- Assume $V = \{1, 2, \dots, n\}$
- An *adjacency matrix* represents the graph as a $n \times n$ matrix A :
 - $A[i, j] = 1$ if edge $(i, j) \in E$ (or weight of edge)
 $= 0$ if edge $(i, j) \notin E$
 - Storage requirements: $O(V^2)$
 - A dense representation
 - But, can be very efficient for small graphs
 - Especially if store just one bit/edge
 - Undirected graph: only need one diagonal of matrix

Universal Sink

- Show how to determine whether a directed graph G contains a universal sink - a vertex with in-degree $(V-1)$ (V is the number of vertices) and out-degree 0, given an adjacency matrix for G . Can be done in $O(V)$

Review: Breadth-First Search

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its children, then their children, etc.

Review: Breadth-First Search

- Again will associate vertex “colors” to guide the algorithm
 - White vertices have not been discovered
 - All vertices start out white
 - Grey vertices are discovered but not fully explored
 - They may be adjacent to white vertices
 - Black vertices are discovered and fully explored
 - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

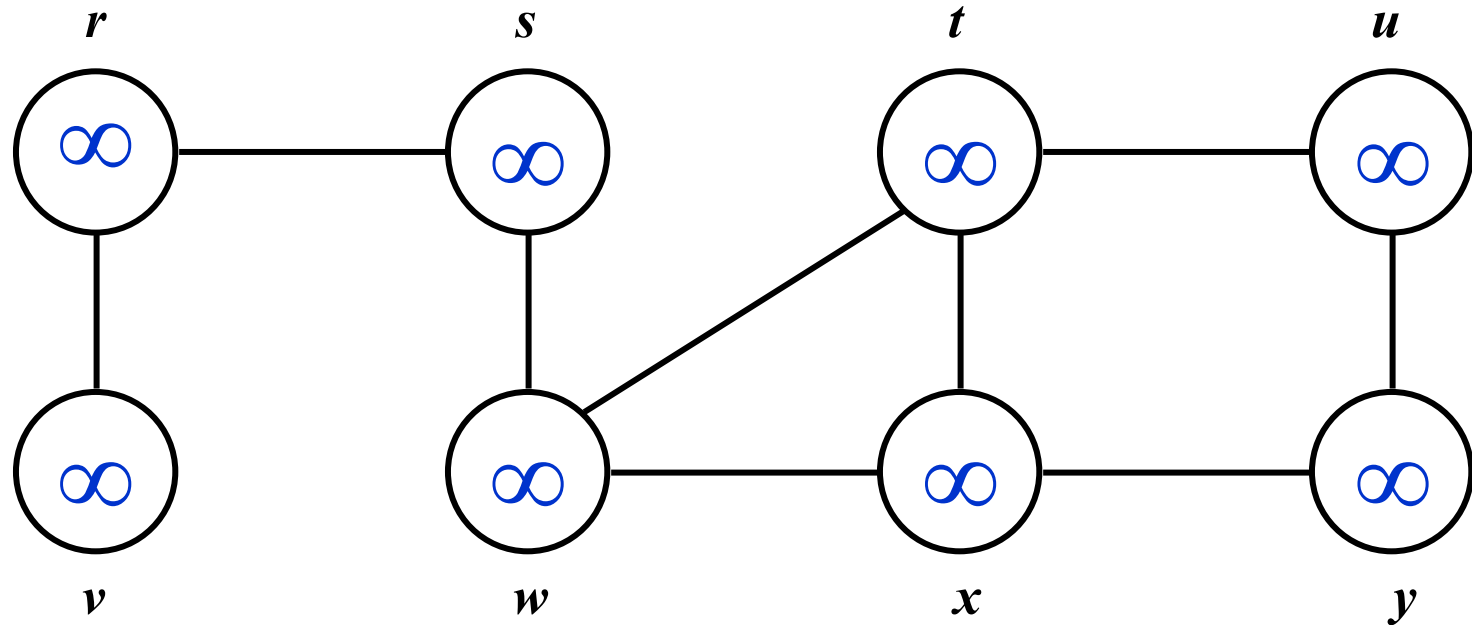
Review: Breadth-First Search (assumes connected Graph)

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};           // Q is a queue (duh); initialize to s  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;      v->d represents level of v  
                v->p = u;           v->p represents the parent of v  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

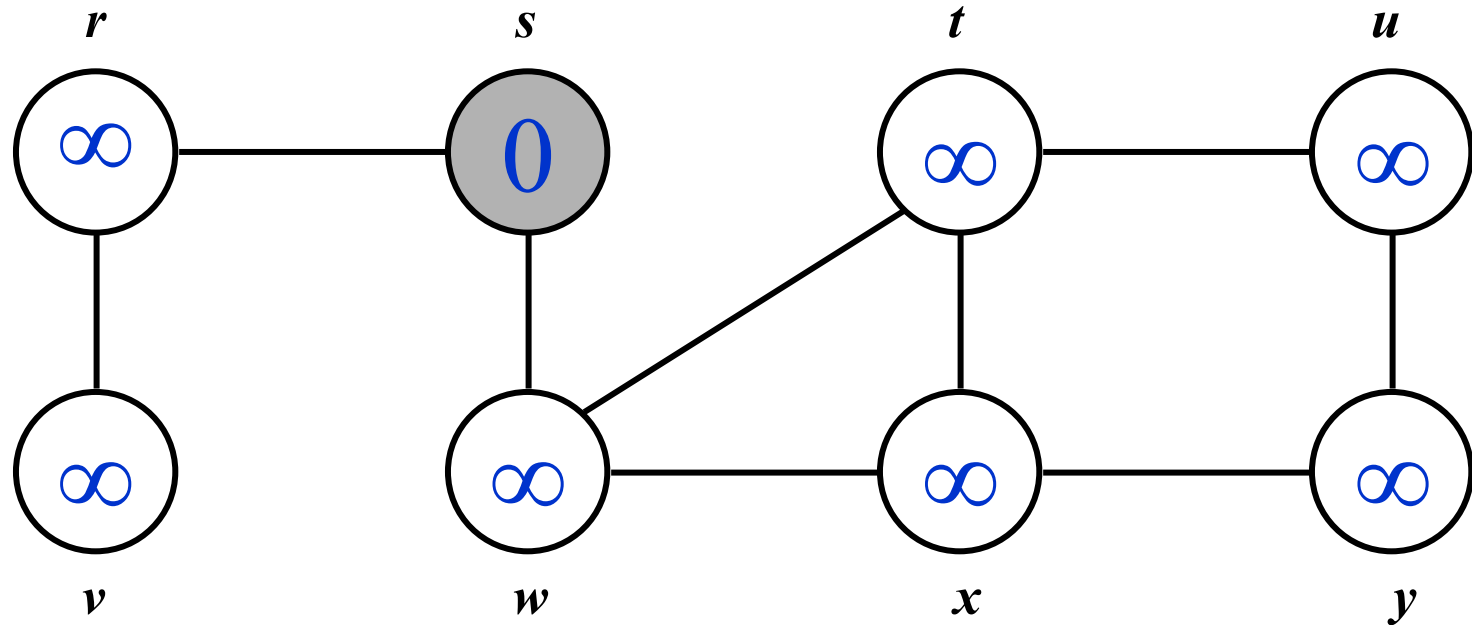
Homework

Write down the BFS code for when the graph can be disconnected

Breadth-First Search: Example

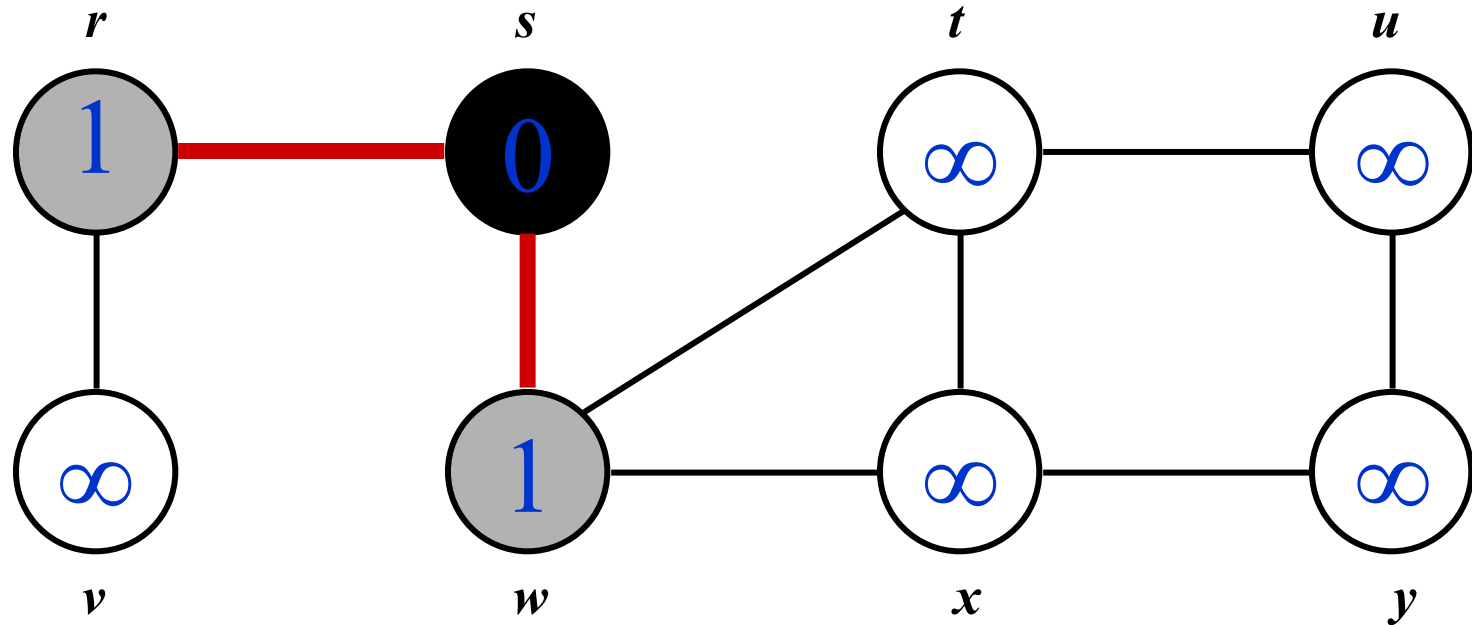


Breadth-First Search: Example



Q : s

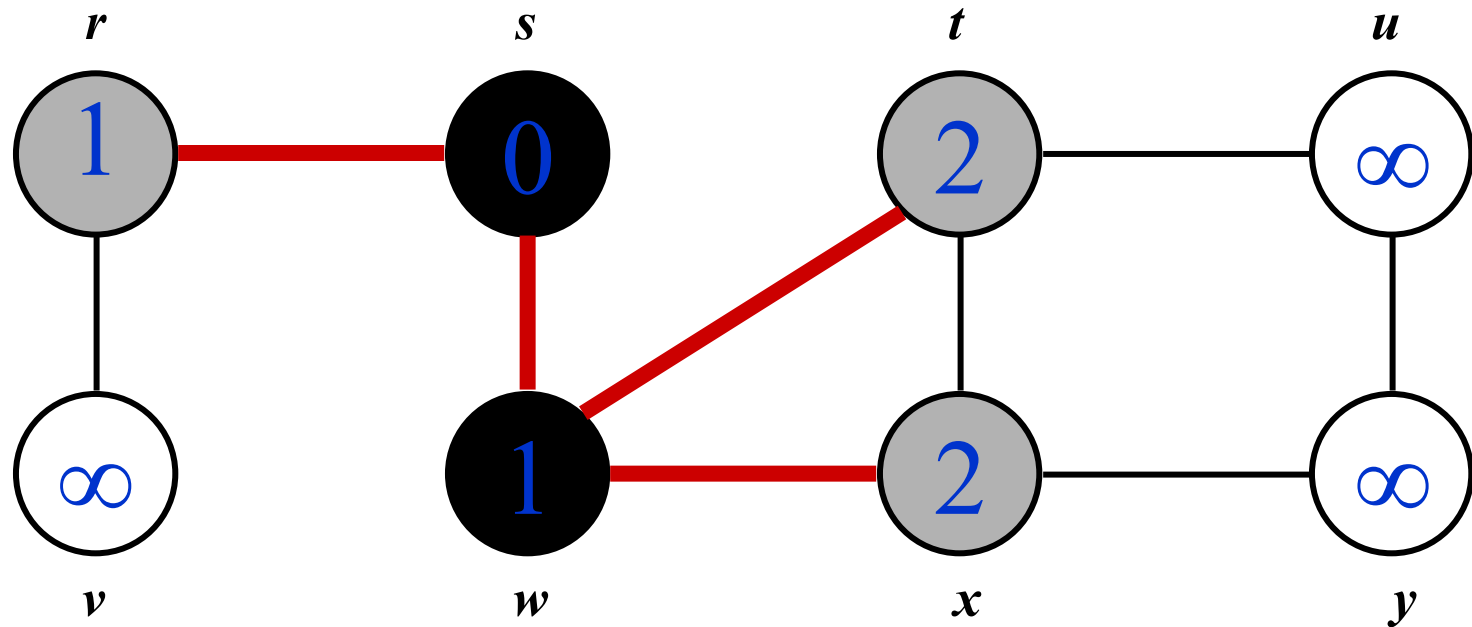
Breadth-First Search: Example



Q :

w	r
-----	-----

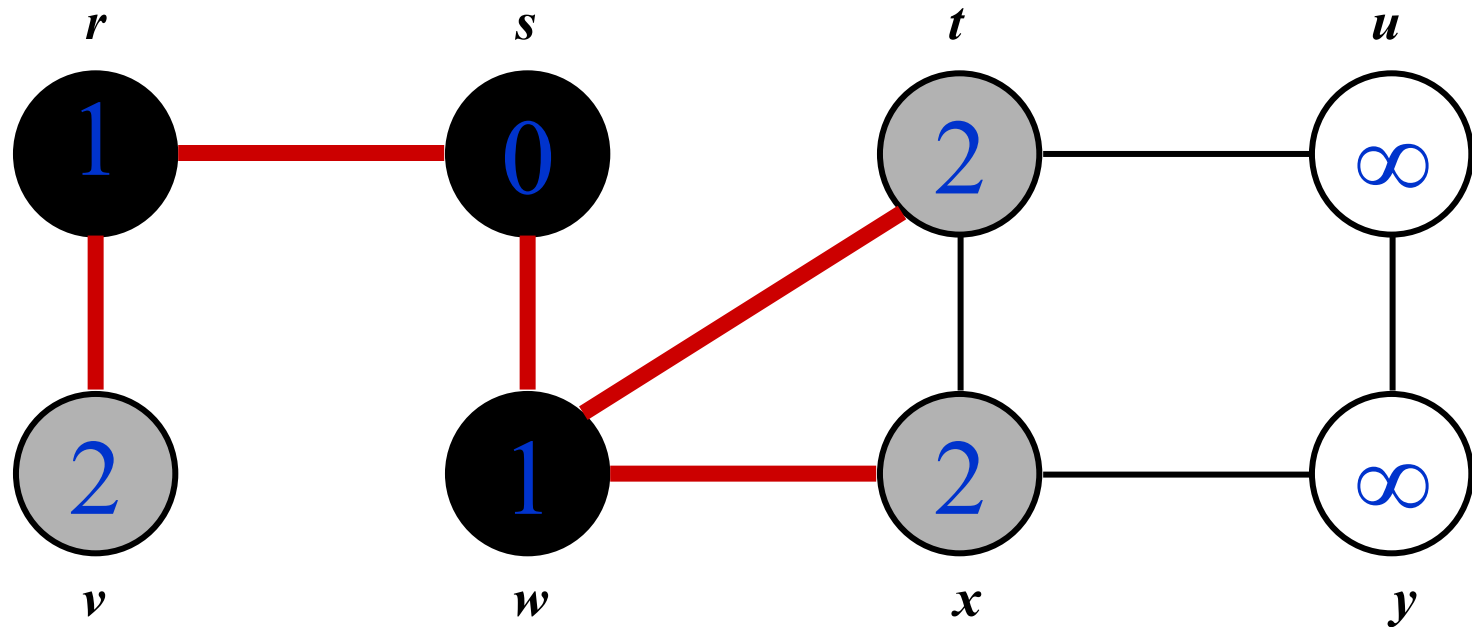
Breadth-First Search: Example



Q :

r	t	x
-----	-----	-----

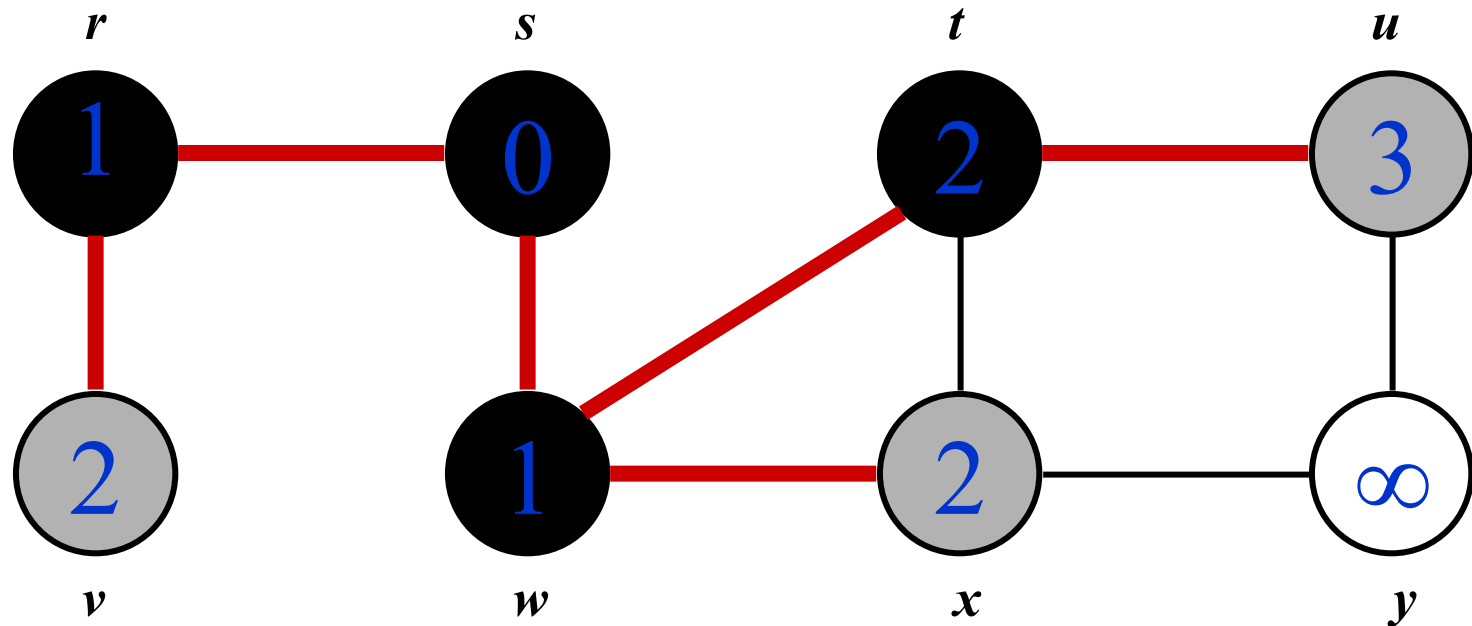
Breadth-First Search: Example



Q :

t	x	v
-----	-----	-----

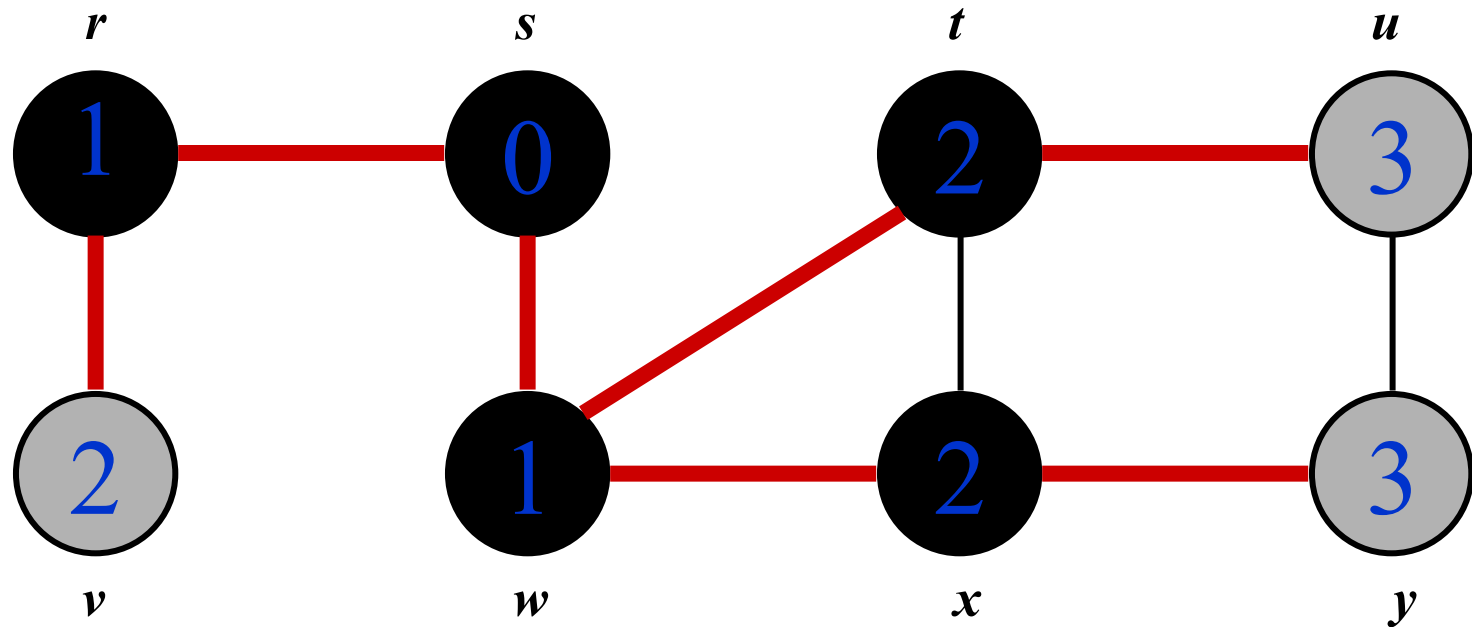
Breadth-First Search: Example



Q :

x	v	u
-----	-----	-----

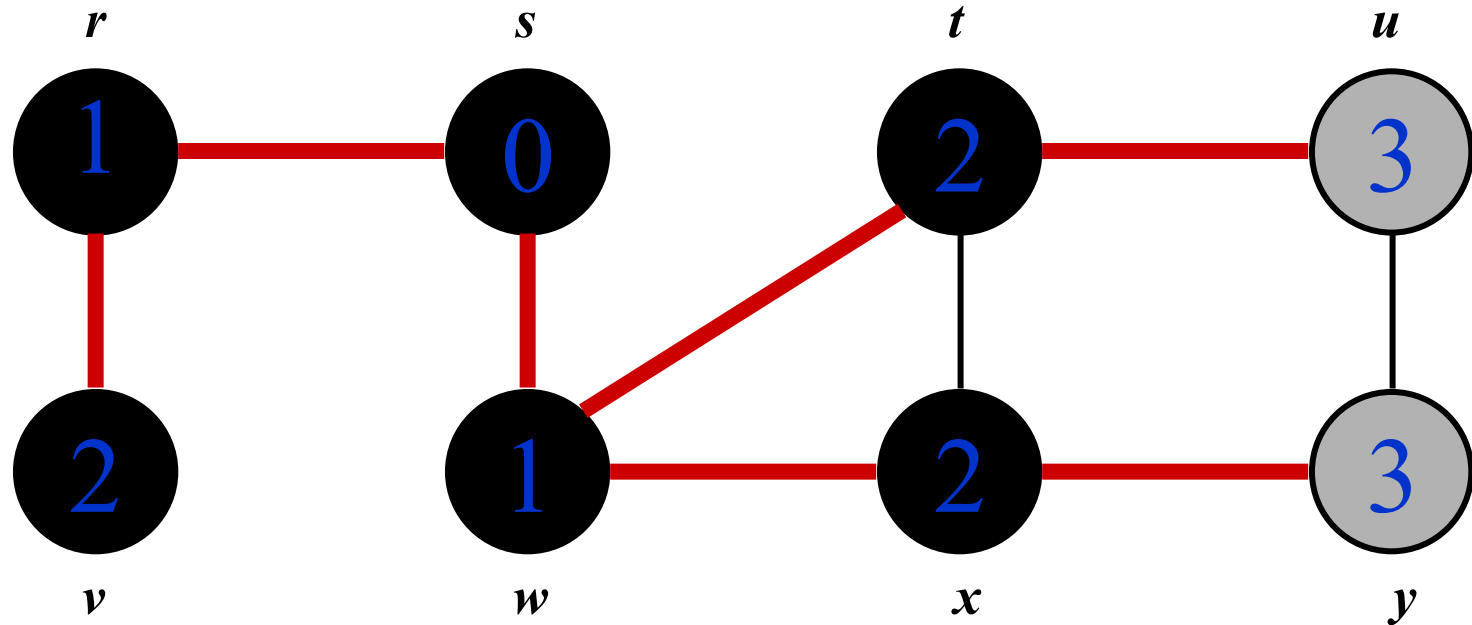
Breadth-First Search: Example



Q :

v	u	y
-----	-----	-----

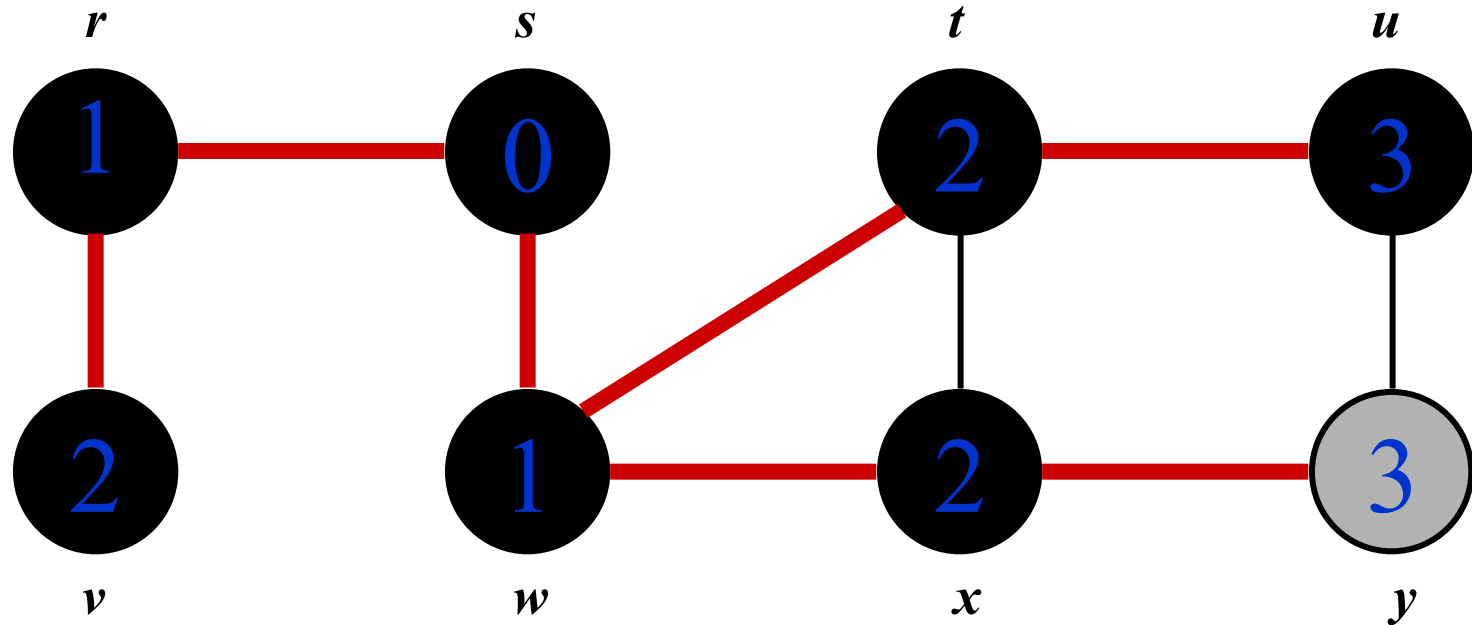
Breadth-First Search: Example



Q :

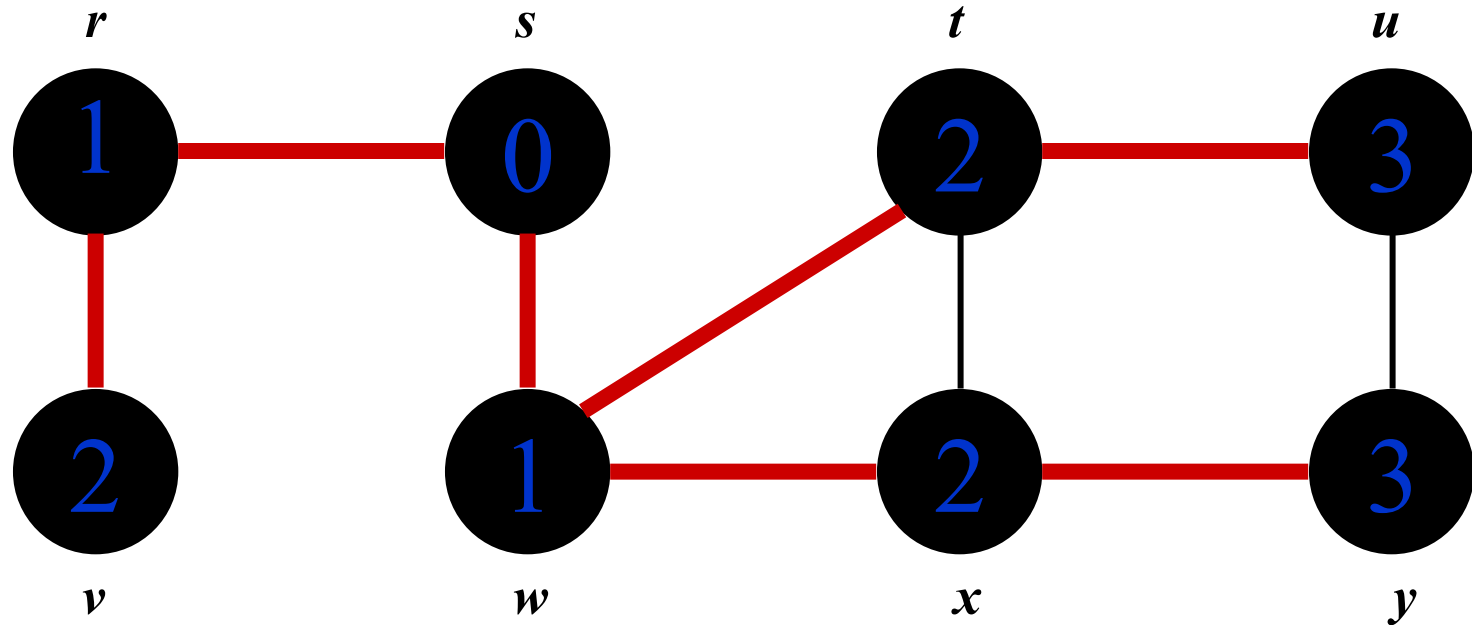
u	y
-----	-----

Breadth-First Search: Example



Q : y

Breadth-First Search: Example



$Q: \emptyset$

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  Touch every vertex:  $O(V)$   
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  u = every vertex, but only once  
        for each v  $\in$  u->adj { (Why?)  
             So v = every vertex that appears in some other vert's adjacency list if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
            }  
        u->color = BLACK;  
    }  
}
```

What will be the running time?

Total running time: $O(V+E)$

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
- Since BFS gives a path from root to node, $d[v] \geq sd[v]$ (can't be shorter than shortest path)
- To prove that $d[v] \leq sd[v]$
- $D[v]$ is distance reported by bfs, $sd[v]$ is shortest possible distance to v from s

Proof sketch

- Let v be node closest to s that has $d[v] \neq sd[v]$.
- Now $d[v]$ can't be less than $sd[v]$ as $sd[v]$ is the shortest possible distance. Hence to prove $d[v] > sd[v]$.
- Consider the actual shortest path that bfs has missed. Let u be vertex just before v on shortest path from s to v which means $sd[v] = sd[u] + 1$
- Lets look at what could have happened during the bfs when u was being explored. Note u is grey.
 - either v was white or black or grey. All show contradiction
 - If v was white: then $d[v] = d[u] + 1$
 - If v was black: then v was enqueued before u so $d[v] \leq d[u]$
 - If v was grey: so v was discovered through say w . Hence $d[w]$ was enqueued before u and hence $d[w] \leq d[u]$. Hence, $d[w] + 1 \leq d[u] + 1$. Hence $d[v] = d[w] + 1 \leq d[u] + 1$.

Problems

- Given an undirected graph, report if the graph contains a cycle.
- Count the number of connected components in an undirected graph.
- Can you find source vertex of a connected directed graph using BFS
- Report whether a given undirected graph is bipartite.
(A bipartite graph is a set of graph vertices partitioned into two disjoint, independent sets U , V such that every edge connects a node in U to a node in V and no edges exist between vertices in the same set)

Compute the diameter of the undirected tree using BF.

Diameter: The maximum distance between any two vertices in the tree, where distance is measured by the number of edges in the unique simple path connecting them.

